



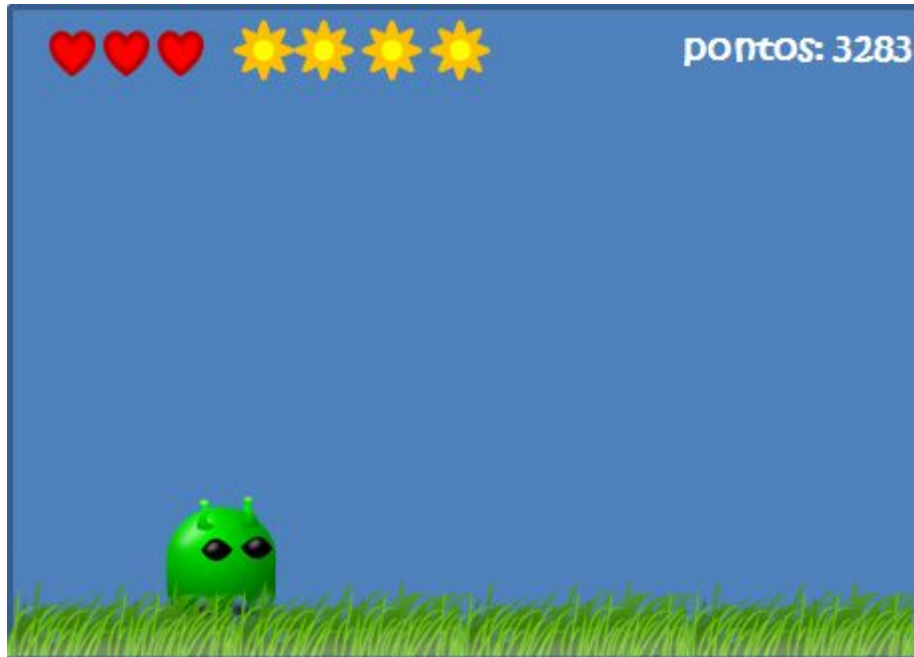
Introdução a Técnicas de Programação

Tipos de variáveis

Prof. André Campos
DIMAp/UFRN

Variáveis

Informações armazenadas na memória durante a execução de um programa



pontos
número de vidas
quantidade de itens
posição
...

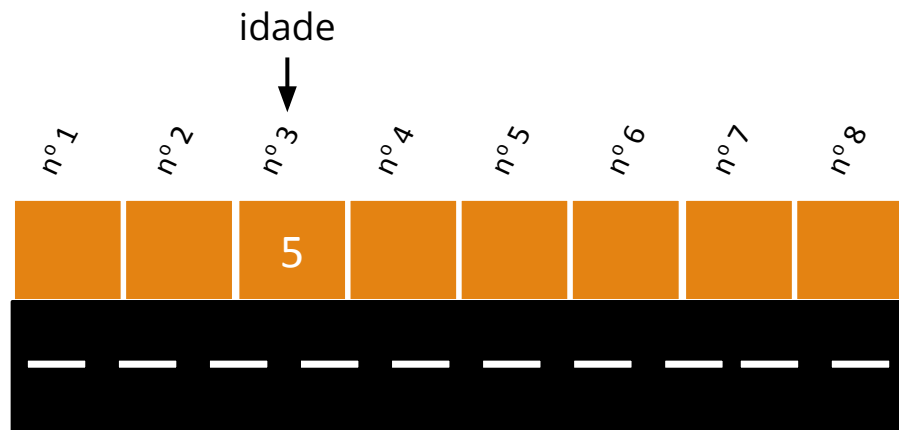
Endereços da memória do computador

A memória é organizada como uma **sequência de blocos (bytes)**, cada um com seu endereço (posição na memória).



Variáveis na memória

Referências para acessarmos ou guardarmos os valores nos endereços da memória. É como um “apelido” de um endereço na memória.





Tipos, valores e identificadores

Em C++, cada variável possui:

- Um **tipo**: define o que é armazenado e quantos números de endereços serão usados.
- Um **identificador**: nome da variável (apelido do endereço)
- Um **valor**: a informação nos endereços
 - Só esse pode ser alterado durante a execução do programa

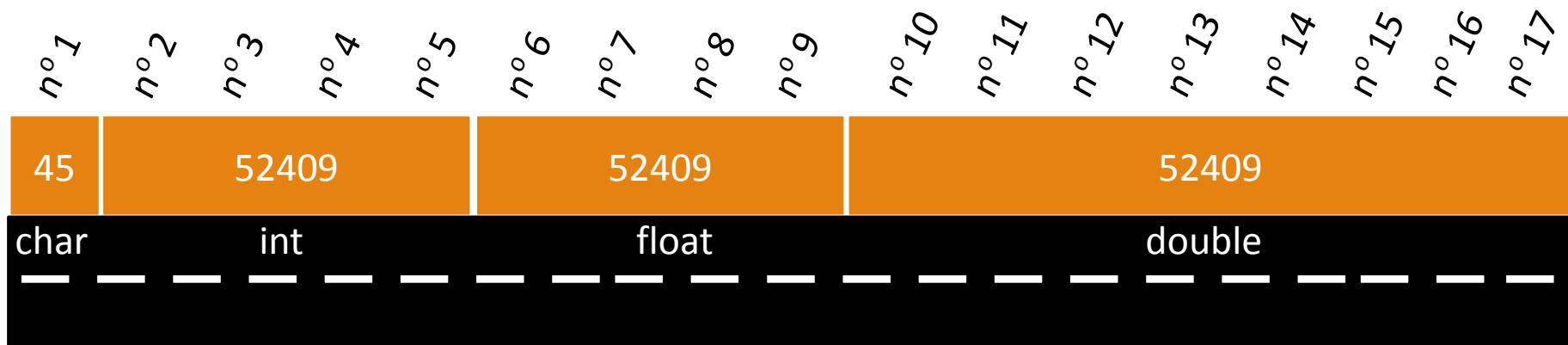
Os identificadores podem conter letras, números e "_", mas não pode começar com número.

Ex: pontos_vida

Alguns tipos básicos (primitivos)

Em cada número de endereço, "reside" um valor de 1 byte (8 bits).

- **char**: reside um número inteiro representado por 1 byte.
- **int**: reside um número inteiro representado por 4 bytes.
- **float**: reside um número real representado por 4 bytes.
- **double**: reside um número real representado por 8 bytes.





Tipo char

Cada tipo define os endereços a ocupar e como será “interpretado”

- **char: [-128; 127]**

- 00000000 = 0
- 00000001 = 1
- 00000010 = 2
- ...
- 01111110 = 126
- 01111111 = 127
- 10000000 = -128
- 10000001 = -127
- 10000010 = -126
- ...
- 11111111 = -1

- **unsigned char: [0; 255]**

- 00000000 = 0
- 00000001 = 1
- 00000010 = 2
- ...
- 01111110 = 126
- 01111111 = 127
- 10000000 = 128
- 10000001 = 129
- 10000010 = 130
- ...
- 11111111 = 255

Tipo char e a tabela ASCII

Valores de 0 a 31 correspondem a caracteres de controle (salto de linha, tab...)

LET	VAL
espaço	32
!	33
"	34
#	35
\$	36
%	37
&	38
'	39
(	40
)	41

LET	VAL
*	42
+	43
,	44
-	45
.	46
/	47
0	48
1	49
...	...
9	57

LET	VAL
:	58
;	59
<	60
=	61
>	62
?	63
@	64
A	65
...	...
Z	90

LET	VAL
[	91
\	92
]	93
^	94
_	95
`	96
a	97
b	98
...	...
z	122

LET	VAL
{	123
	124
}	125
~	126
delete	127



Tipo int

int reserva 4 bytes

2^{32} possíveis valores

long reserva 8 bytes

2^{64} possíveis valores

short reserva 2 bytes

2^{16} possíveis valores

Pode-se usar o modificador

unsigned (sem sinal)

- **int:** [-2.147.483.648; 2.147.483.647]
 - 00000000 00000000 00000000 00000000 = 0
 - 00000000 00000000 00000000 00000001 = 1
 - ...
 - 11111111 11111111 11111111 11111111 = -1
- **unsigned int:** [0; 4.294.967.295]
 - 00000000 00000000 00000000 00000000 = 0
 - 00000000 00000000 00000000 00000001 = 1
 - ...
 - 11111111 11111111 11111111 11111111 = 4.294.967.295
- **long** (ou long int)
- **short** (ou short int)
- **unsigned long**
- **unsigned short**

Tipos “ponto flutuante”

float: possui 4 bytes (32 bits) para expressar valores numéricos reais (ex: 5,3)

double: similar, mas com 8 bytes (64 bits) para expressar valores maiores e com maior precisão

Tipos “booleano”

bool: pode assumir apenas 2 valores: **true** ou **false**



George Boole (1815-1864)



Declarando e usando variáveis

Para reservar um espaço na memória, é necessário declarar uma variável

```
int x;
```

Podemos declarar uma lista de variáveis de um mesmo tipo

```
float x, y, z;
```

Podemos inicializar seu valor com uma constante ou uma expressão

```
bool x = true;  
int y = 2 * 10;
```

Há também outras formas de inicializar em C++

```
int a { 10 };  
int b ( 4 );
```



Operadores matemáticos

Operadores são usados em expressões para combinar valores

- $- a$: retorna o inverso de a (multiplicação por -1)
- $a + b$: retorna a soma de a com b
- $a - b$: retorna a subtração a por b
- $a * b$: multiplica a por b
- a / b : divide a por b e retorna o quociente
- $a \% b$: divide a por b e retorna o resto da divisão

ATENÇÃO: o resultado da operação pode exceder os limites aceitos pelo tipo.

- Transbordamento (ou *overflow*) pode ocorrer e deve ser evitado.
- A linguagem C++ não possui mecanismos para detectar *overflow*
- O programador deve portanto sempre verificar os limites aceitos pelo tipo



Operador de atribuição

Na atribuição, primeiro a expressão à direita do operador é calculada e depois o valor é atribuído. Então, esta é uma instrução válida:

```
x = x + 1;
```

O valor da variável `x` é incrementado de 1. Há operadores especiais para alterar o valor de uma variável baseado no seu valor atual:

```
x += 1; // aumenta o valor de x em uma unidade  
x++;   // faz o mesmo
```

Quando um valor de um tipo é atribuído a uma variável de outro, pode haver uma conversão automática de tipos. Ex:

```
int x = 5.4; // x vale 5!
```



Mistério

Além de +=, há operadores semelhantes para outras operações

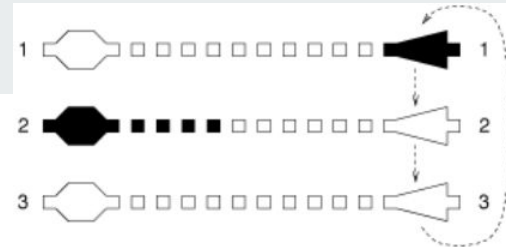
+= -= *= /= %=

Considere o seguinte trecho de programa

```
int x = a + b;  
int y = x - b;  
x -= y;
```

Ao final da execução desse trecho, quais os valores de x e de y (expressos em termos de a e b)?

Tarefa: Game-10 (OBI 2017)



No princípio dos anos 1980 surgiram nos colégios os primeiros relógios de pulso digitais com joguinhos. Era uma febre entre os alunos e quem tinha um era muito popular na hora do recreio. Os joguinhos eram bem simples, mas muito legais. Um dos primeiros era o Game-10, no qual você controlava um avião que aparecia na parte direita do visor. Na parte esquerda aparecia um disco voador em qualquer uma de três posições, aleatoriamente, e lançava um míssil. O objetivo do jogador era movimentar o avião verticalmente para que ficasse na frente do disco voador (na mesma linha horizontal, do lado direito) e atirar para interceptar o míssil antes que esse atingisse o avião.

Como o movimento do avião era feito com apenas um botão, só dava para movimentar em um sentido: ao apertar o botão sucessivas vezes, o avião se movia na sequência de posições ... $1 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 1 \dots$

Veja que, na situação da figura, o jogador deveria apertar o botão apenas uma vez, para ir da posição 1 para a posição 2, e conseguir atirar e interceptar o míssil.

Tarefa: Game-10 (OBI 2017)

Neste problema vamos considerar que existem N posições e não apenas três. Dado o número de posições N , a posição D na qual o disco voador aparece, e a posição A onde está o avião, seu programa deve computar o número mínimo de vezes que o jogador precisa apertar o botão para movimentar o avião até a mesma posição do disco voador e poder atirar!

Entrada

A primeira linha da entrada contém um inteiro N , o número de posições. A segunda linha contém um inteiro D , a posição do disco voador. A terceira linha contém um inteiro A , a posição do avião.

Saída

Seu programa deve imprimir uma linha contendo um inteiro, o número mínimo de vezes que o jogador deve apertar o botão para poder atirar.



Exemplos de entrada e saída

Entrada	Saída
3 2 1	1
5 1 5	1
20 8 13	15

Obs: Não usem condicionais! (if)

Dica: Use o operador de resto de divisão (%)



Considerações sobre o tipo char

Um caracter é representado tanto por um valor numérico (seu código ASCII) quanto pela sua representação gráfica entre aspas simples. Ex: 'A'.

Assim, podemos inicializar uma variável do tipo char com sua representação gráfica. Abaixo, `a` e `b` possuem valores iguais.

```
char a = 'A';  
char b = 65;
```

Podemos inclusive realizar operações com eles.

```
char a = 'S' - 'A' + 'a';
```



Atividade

Crie um programa que lê da entrada uma letra minúscula (considere que o usuário irá sempre digitar uma letra minúscula) e envie para a saída a letra maiúscula correspondente.

Exemplos de entrada e saída

Entrada	Saída
h	H
a	A
z	Z
i	I



Considerações sobre operações sobre inteiros

ATENÇÃO: Operações com variáveis de um tipo, resultam em um valor do mesmo tipo!!!

Exemplo. Qual o valor de x para cada trecho de código abaixo

```
int x = 7 / 2;  
float x = 7 / 2;
```

Esta é uma fonte de erro frequente. Para termos valores em real (ponto flutuante), precisamos que pelo menos um dos operandos seja ponto flutuante (float ou double).

```
float x = 7 / 2.0;
```



Atividade

Crie um programa que lê da entrada um número real (double) e envia para a saída duas linhas.

A primeira linha deve conter o valor da parte inteira do número lido, enquanto a segunda linha deve conter a parte fracionária com precisão de dois dígitos.

Obs: a parte fracionária não deve ser arredondada.

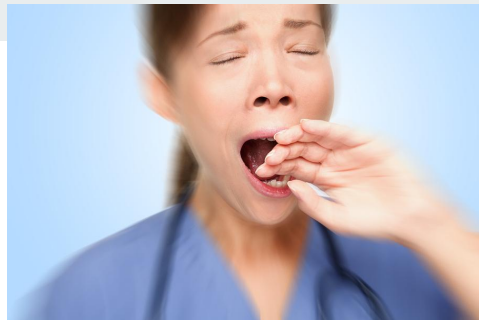
Exemplos de entrada e saída

Entrada	Saída
1.12	1 12
1000.25	1000 25
34.2317	34 23
5.001	5 00

Tarefa: o sono da enfermeira

Daniela é enfermeira em um grande hospital, e tem os horários de trabalho muito variáveis. Para piorar, ela tem sono pesado, e uma grande dificuldade para acordar com despertadores.

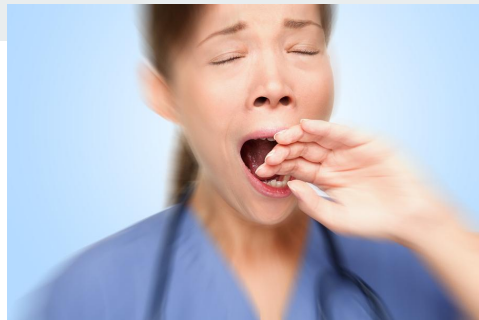
Recentemente ela ganhou de presente um relógio digital e tem esperança que isso resolva o seu problema. No entanto, ela anda muito cansada e quer aproveitar cada momento de descanso. Por isso, carrega seu relógio digital para todos os lugares e tenta dormir sempre que tem um tempo de descanso, programando o alarme do relógio para tocar na hora em que tem que acordar. No entanto, com tanta ansiedade para dormir, acaba tendo dificuldades para adormecer e aproveitar o descanso.



Tarefa: o sono da enfermeira

O problema que a tem atormentado na hora de dormir é saber quantos minutos ela teria de sono se adormecesse imediatamente e acordasse somente quando o despertador tocasse. Porém, ela não é muito boa com números e pediu sua ajuda para escrever um programa que, dada a hora atual (horas e minutos) e a hora do alarme (horas e minutos), determine o número de minutos que ela poderia dormir.

Obs: considere que a hora do alarme é sempre maior que a atual e que nunca passa da meia-noite. Considere também as horas no formato 24h.





Tarefa: o sono da enfermeira

Entrada

A entrada consiste em duas linhas. A primeira linha representa a hora atual e a segunda linha a hora do despertador. Cada linha contém dois valores inteiros indicando respectivamente a hora e o minuto correspondente.

Saída

Seu programa deve enviar para a saída um único valor inteiro com a diferença em minutos da hora do alarme e a hora atual.

Entrada	Saída
11 30 12 00	30
17 52 18 39	47
8 16 23 4	888